Project Execution Guide

Secure Decentralized Key Management Framework

Debian setup and multi-device LAN execution
Updated 5 November 2026

This guide explains how to install the current Python application, connect separate LAN devices and verify that communication and key updates continue after the discovery bootstrap is stopped. Each peer owns its identity, ledger and direct TCP listener. Use the same updated source on every device.

Verification status: automated local tests passed. The team reported successful LAN operation of the earlier version; record a new LAN run for this direct-mesh version using the procedures below.

Debian LAN guide: requirements and device layout

Follow this guide on Debian 12 or 13 with Python 3.11 or newer. Python 3.11+ is required for the supplied regression tests, which use asyncio.timeout().

All commands labelled bash run in a Linux terminal. Commands labelled text run inside the peer CLI, after its [ACTIVE] (...) > prompt appears.

Two devices are enough to test a real LAN connection. Use three peers to test the scoped revocation and lazy verification scenario. This example puts one peer on each of three Debian devices:

Device	Example LAN IPv4	Processes	Terminals
A	192.168.1.100	Relay + Node 1 (Alice)	Two
B	192.168.1.101	Node 2 (Bob)	One
C	192.168.1.102	Node 3 (Charlie)	One

Replace these example addresses with your devices' actual addresses. A separate relay-only device also works: point every peer at that device's IPv4 address.

Requirements:

- Devices on a LAN that permits connections between clients, using Ethernet or Wi-Fi. A guest Wi-Fi network or AP/client isolation can block these connections.
- The same current Python source files on every device.
- Python 3.11+, venv, and cryptography>=42.0.0, as declared in p2p-simple/requirements.txt.
- Administrator access for Debian package installation and any required firewall rule. Run the relay and peers as your normal user.
- Internet access for the initial APT/PyPI installation. The running application uses LAN TCP connections and does not need an external blockchain or RPC service.
- Inbound TCP 4001 allowed on the bootstrap device during discovery.
- Inbound TCP 5001 allowed on every peer device for the commands below. Peers need outbound access to one another's direct listening ports as well as the optional bootstrap. Multiple peers on one computer need distinct listening ports; peers on different devices can all use 5001.
- A separate identity file for each peer and an interactive terminal for its CLI.

peer.py launches peer_cli.py; both accept the same options. --port 8501 selects peer_identity_8501.json; it does not open TCP port 8501.

--listen-port 5001 is the actual peer TCP listener. Omitting --listen-port

chooses an available port, which is convenient for local tests but harder to configure in a LAN firewall.

1. Install Debian packages on every device

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip ca-certificates unzip iproute2 iputils-ping netcat-openbsd
python3 --version
```

Confirm the version is 3.11 or newer. If your account cannot use sudo, ask the administrator to install these packages, then continue as your normal user.

2. Copy the current source to every device

The current working copy may contain fixes that have not been committed and pushed. Copy the current source when testing those fixes; a clone of the remote repository only contains its published commits.

Copy from the current Windows working folder

On the Windows source computer, open PowerShell in the repository root (C:\Users\dalme\Documents\minor\Minor) and run:

```
@'
from pathlib import Path
from zipfile import ZIP_DEFLATED, ZipFile


files = [Path("README.md"), Path("p2p-simple/requirements.txt")]
files.extend(sorted(Path("p2p-simple").glob("*.py")))
with ZipFile("minor-lan-source.zip", "w", compression=ZIP_DEFLATED) as archive:
    for path in files:
        archive.write(path, path.as_posix())
print("Created minor-lan-source.zip")
'@ | python -
```

This includes the Python source, tests, requirements, and README, with the p2p-simple/ directory structure intact. Identity JSON files, private keys, virtual environments, and node_modules are excluded by the explicit file list.

Copy minor-lan-source.zip to your home directory on each Debian device using a USB drive, shared folder, or your usual file transfer method. On each device:

```
mkdir -p ~/minor-lan
unzip ~/minor-lan-source.zip -d ~/minor-lan
cd ~/minor-lan/p2p-simple
ls peer.py peer_cli.py peer_node.py relay.py blockchain.py crypto.py crypto_modular.py requirements.txt
```

For later source updates, stop your peer process before extracting the new archive. Preserve its identity JSON file to keep the same peer ID.

Copy from an existing Linux working folder

From that repository's root, create an equivalent source archive:

```
tar -czf minor-lan-source.tar.gz README.md p2p-simple/*.py p2p-simple/requirements.txt
```

Copy it to each device's home directory, then extract it:

```
mkdir -p ~/minor-lan
tar -xzf ~/minor-lan-source.tar.gz -C ~/minor-lan
cd ~/minor-lan/p2p-simple
```

Use Git when the required changes have been published

```
sudo apt install -y git
git clone https://github.com/errol-dalmeida/Minor_project.git ~/minor-lan
cd ~/minor-lan/p2p-simple
```

Choose one source-copy method. Use the same version on all devices. Each peer

generates its own identity on first launch; copying another peer's identity file

would cause both processes to claim the same peer ID.

3. Create the Python environment on every device

From ~/minor-lan/p2p-simple:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip check
python -c "import sys, cryptography; from peer_node import PeerNode; print(sys.version); print('cryptography',
cryptography.__version__); print('PeerNode imports OK')"
python peer.py --help
```

Install application packages inside this environment. Debian's system Python is

externally managed; a virtual environment provides an isolated Python and pip.

See Debian's Python virtual environment instructions.

Every new terminal needs these two commands before launching the application:

```
cd ~/minor-lan/p2p-simple
source .venv/bin/activate
```

If the source has been copied to another location, adjust the cd path. Keep

running a peer from the same directory so it finds the same relative identity file.

Most compatible Linux installations can install cryptography from a wheel

without compilers after updating pip. If pip must compile it, install the build

packages and check the Rust version:

```
sudo apt install -y build-essential libssl-dev libffi-dev python3-dev cargo pkg-config
rustc --version
cargo --version
python -m pip install -r requirements.txt
```

Debian 12's packaged Rust may be too old for recent cryptography releases.

If installation reports an unsupported Rust version, install the toolchain
required by the cryptography installation guide
and retry inside .venv. These build packages are only needed for a source build.

4. Find every device's LAN address and allow discovery and peer TCP ports

On every device:

```
ip -br -4 address
ip -4 route
```

Choose the IPv4 address on the Ethernet/Wi-Fi interface used by Devices B and C,
for example 192.168.1.100 on Device A. Peers use Device A's address in --relay,
and their own device's address in --advertise-host. A remote peer cannot reach
your listener at 127.0.0.1; that address is for single-computer tests.
Keep the device addresses stable during the test. A router DHCP reservation is
useful for repeat runs; otherwise recheck the address after reconnecting Wi-Fi.
For peers or the bootstrap inside VMs, configure reachable bridged LAN interfaces
or forward the corresponding TCP listening ports through their VM hosts.
If UFW already manages the devices, allow direct connections on every peer:

```
sudo ufw status verbose
sudo ufw allow proto tcp from 192.168.1.0/24 to any port 5001 comment 'P2P direct peers LAN'
sudo ufw status numbered
```

On Device A, also allow bootstrap discovery:

```
sudo ufw allow proto tcp from 192.168.1.0/24 to any port 4001 comment 'P2P bootstrap LAN'
```

Replace 192.168.1.0/24 with your actual LAN subnet. If you choose UFW for a
host with no existing firewall manager, install it with sudo apt install -y ufw
and add the rule above. Enabling a new UFW firewall is optional for this test;
if you enable it over SSH, allow your SSH port first, for example:

```
sudo ufw allow 22/tcp
sudo ufw enable
```

Use your actual SSH port if it differs from 22. UFW activation changes the host's
firewall policy; Debian's UFW manual
explains remote-management rules. An inactive UFW instance does not enforce its
rules. If nftables or firewalld already manages this device, use that manager to
allow LAN clients to reach TCP 5001 on peers and TCP 4001 on the bootstrap host.
On hosts with restricted outbound rules, also allow peer connections to the
bootstrap's IPv4 address on TCP 4001 and other peers on TCP 5001.

No router Internet port forwarding is needed

for devices that can already reach each other on the LAN.

5. Start a discovery bootstrap on Device A

In Device A's first terminal:

```
cd ~/minor-lan/p2p-simple
source .venv/bin/activate
RELAY_HOST=0.0.0.0 RELAY_PORT=4001 python relay.py
```

Expected output includes Relay running and addresses ending in /tcp/4001/....

Keep this terminal running while the initial peers discover each other. It can

be stopped once direct connections are established. Peers take IPv4:port, such as

192.168.1.100:4001, rather than the full printed /ip4/.../p2p/... string.

In another terminal on Device A, confirm the listener:

```
ss -ltnp 'sport = :4001'
```

It should show port 4001 listening on 0.0.0.0.

6. Check connectivity from Devices B and C

On each remote device, while the relay is running:

```
ping -c 3 192.168.1.100
nc -4 -vz -w 3 192.168.1.100 4001
```

Replace the example IP with Device A's address. The TCP check should report

succeeded. Ping can be filtered independently, so the TCP check is the relevant

test for this application. See the Debian netcat manual.

Resolve a refused or timed-out TCP connection using the troubleshooting table

below before starting peers.

7. Start one peer on each device

On Device A, in its second terminal:

```
cd ~/minor-lan/p2p-simple
source .venv/bin/activate
umask 077
python peer.py --port 8501 --listen-port 5001 --advertise-host 192.168.1.100 --relay 192.168.1.100:4001
```

On Device B:

```
cd ~/minor-lan/p2p-simple
source .venv/bin/activate
umask 077
python peer.py --port 8502 --listen-port 5001 --advertise-host 192.168.1.101 --relay 192.168.1.100:4001
```

On Device C:

```
cd ~/minor-lan/p2p-simple
source .venv/bin/activate
umask 077
python peer.py --port 8503 --listen-port 5001 --advertise-host 192.168.1.102 --relay 192.168.1.100:4001
```

Replace all example IPs with your actual device addresses. umask 077 makes newly created identity files accessible to their owner. Each terminal should print its peer ID, Peer listening, and a CLI prompt. Discovery events appear in the background.

Allow the direct connections and initial key and chain exchanges to finish, then enter these commands inside every peer CLI:

```
status
peers
trust
crypto sha256 aes-gcm
```

Each peer should list two other connected peers. Copy the full peer IDs into your test notes as Node1PeerId, Node2PeerId, and Node3PeerId. Replace every <Node...PeerId> placeholder below with that actual ID; omit the angle brackets.

Keep each CLI in the foreground. Peers start listening even when the bootstrap is unavailable and retry discovery in the background. Existing direct links use heartbeats and reconnect to known addresses independently of the bootstrap.

From another device, check each running peer's listener, for example:

```
nc -4 -vz -w 3 192.168.1.101 5001
```

For several peers on one computer, use --advertise-host 127.0.0.1 and different --listen-port values, such as 5001, 5002, and 5003.

Join through an existing peer when the bootstrap is stopped

A new or restarted peer can contact any known running member. For example, after stopping Node 3, restart it on Device C using Node 1's peer port, not port 4001:

```
python peer.py --port 8503 --listen-port 5001 --advertise-host 192.168.1.102 --no-relay --peer 192.168.1.100:5001
```

The contacted peer supplies other live peer addresses; Node 3 connects to them directly. Repeat --peer HOST:PORT to supply more seed addresses. A fresh network can also start without any bootstrap: start the first peer with --no-relay, then give its address to subsequent peers. A newcomer needs either a reachable bootstrap or at least one reachable peer address.

8. Run the tests across the actual LAN devices

Run the scoped/lazy test first, before any successful DM between Nodes 2 and 3.

DM history makes peers active contacts, which changes which peers receive later

scoped notifications. The supplied automated tests run on localhost; the following manual steps verify the actual devices and LAN connection.

Test A: scoped revocation, lazy verification, and reinstatement

Inside Node 1 and Node 2:

```
join #dev
leave #general
channels
```

Inside Node 3:

```
join #marketing
leave #general
channels
metrics
```

Node 1 and Node 2 should now belong to #dev; Node 3 should belong to #marketing. Sharing the default #general channel would make all three peers contacts, so leaving it is part of this test setup.

On Node 2:

```
revoke 1
metrics
```

On Node 1 and Node 3, inspect:

```
trust
metrics
```

Expected: Node 1 receives the revocation, marks Node 2 revoked, and sets its trust to zero. Node 3 receives zero pushed revocation packets; it can still show its previous local status until it requests an update. With this fresh setup, Node 2 sends one scoped revocation packet, to Node 1.

On Node 3:

```
dm <Node2PeerId> lazy-check-test
trust
metrics
```

Expected: the DM is blocked; a lazy cache miss triggers an on-demand chain request; Node 2 becomes revoked with trust zero on Node 3. Node 2 should not print the message as a received DM. To check local transmission blocking, on Node 2:

```
#dev this-message-must-be-blocked
broadcast this-broadcast-must-be-blocked
```

Both sends should report a revoked identity. Then reinstate Node 2:

```
unrevoke
```

Retry on Node 3:

```
dm <Node2PeerId> hello-after-reinstatement
trust
metrics
```

Expected: Node 3 refreshes the previously revoked recipient, discovers the

reinstatement, sends the DM, and Node 2 displays hello-after-reinstatement.

For an explicit fresh check, run refresh <Node2PeerId> on Node 3. After this

successful DM, Nodes 2 and 3 are contacts for subsequent scoped updates.

Test B: confidential direct messages

On Node 1:

```
dm <Node2PeerId> private-message-from-device-A
```

Expected: Node 2 prints the verified incoming DM; Node 3 does not print it as an

incoming message. Reply from Node 2:

```
dm <Node1PeerId> reply-from-device-B
```

Node 1 should display the reply. A successful send means the sender wrote the

packet to the direct peer connection; the recipient's displayed message confirms application

delivery and verification.

Test C: channel delivery

On Node 1:

```
#dev hello-dev-team
```

Expected: Node 2 receives the message; Node 3 does not display it because it has

not joined #dev. To test joining the group, on Node 3:

```
join #dev
```

Send another #dev message from Node 1. Both Node 2 and Node 3 should now display

it. Run this step after Test A, since joining #dev changes the contact graph.

Test D: signed public broadcast

On Node 1:

```
broadcast hello-all-LAN-devices
```

Both other nodes should display the verified broadcast regardless of their

channel membership.

Test E: key rotation

On Node 2:

```
status
rotate
status
```

The peer ID should remain the same and its public key should change. On Node 1:

```
refresh <Node2PeerId>
lookup <Node2PeerId>
dm <Node2PeerId> message-after-key-rotation
```

Node 1 should resolve the new key, and Node 2 should receive the DM. chain or

blocks displays the signed lifecycle transactions in each peer's local ledger.

Test F: disconnect and reconnect

On Node 3, enter exit. Nodes 1 and 2 should report that it left. Relaunch Node 3 from its Linux shell using the same command, directory, and identity file as in step 7. Its peer ID should remain the same, and peers should again show two other nodes. Use join #dev again if you want to restore that channel membership.

For the bootstrap-independence test, press Ctrl+C in Device A's relay terminal, leaving all peer terminals running. Keep the bootstrap stopped throughout:

1. Run peers on every node; existing peers and channel memberships must remain.
2. Exchange DMs, channel messages and broadcasts, checking recipient output.
3. Run revoke, verify that contacts receive the update and reject messages, then run unrevoke and verify communication resumes.
4. Run rotate and send another DM using the updated key.
5. Run refresh <peerId> and inspect receipts to check direct synchronization.
6. Restart a peer with --no-relay --peer HOST:PORT as shown above and verify it rejoins.

You may then restart the bootstrap. Its disconnection/reconnection must not clear direct-peer state or duplicate delivered messages. Discovery retries are independent of peer reconnection attempts.

Record the result of Tests A-F, the devices' IPs, peer IDs, Python/cryptography versions, and any failed command or receiver output. This distinguishes a LAN connectivity failure from a key, cache, or messaging failure.

Terminal commands

Command	Action
menu / help	Guided numbered menu / command reference
peers / status	Peer IDs, connections, local identity and ledger status
dm <peerId-or-prefix> <message>	Encrypted, signed direct message
@<peerId-or-prefix> <message>	Alternative direct message syntax
join #dev / leave #dev	Join or leave a channel, including #general
channel #dev / channels	Choose an active channel / list memberships
#dev <message> / <message>	Message a named channel / the active channel
broadcast <message>	Signed public broadcast
revoke [1-4]	Revoke the current identity; notify active contacts
unrevoke / reinstate	Reinstate the current key

Command	Action
rotate	Generate and publish a replacement keypair
lookup <peerId-or-prefix>	Read a local ledger identity record
refresh <peerId-or-prefix>	Request current status, bypassing the cache
trust / metrics	Trust matrix / push packets, cache hits, misses and TTLs
chain / blocks	Inspect ledger blocks and transactions
crypto sha256 aes-gcm	Select SHA-256 and AES-GCM
crypto blake2b chacha20	Select BLAKE2b and ChaCha20-Poly1305
exit / quit	Disconnect and stop the peer

Use crypto settings that match on communicating peers. Cipher/hash changes are not negotiated automatically. A successful send means a direct connection accepted the write; the receiving terminal separately prints the verified incoming message.

Identity persistence and cache behavior

Peer IDs and keys survive restarts in their identity JSON files. Channel memberships, DM history, trust state, cached statuses, and ledgers are in memory. Restarted peers learn ledger state from connected peers. If every peer stops, there is currently no persisted ledger from which to restore the complete revocation and rotation history. Rejoin your test channels after restarting.

For a clean repeat of Test A, exit and relaunch all peers, preserve their identity files, and apply the separate channel setup before sending any DMs. A new identity can instead be created with an unused --id-file path. Treat these files as private key material belonging to the individual peer.

Positive verification entries have a default 300-second TTL. Scoped lifecycle updates immediately refresh known state; unrelated peers can retain an earlier active result until expiry or a forced refresh. An unavailable chain reply cannot create a fresh active entry. This cache policy does not guarantee instantaneous revocation knowledge for every disconnected or unrelated peer.

Channel keys currently derive from public channel names. These are demonstration groups, not access-controlled private rooms. The native ledger and direct mesh are a LAN prototype; the consensus and cryptographic rules have not changed with the discovery-only bootstrap. Initial discovery still needs an address source.

Automated verification

On each Debian device, use another terminal and the environment from step 3:

```
cd ~/minor-lan/p2p-simple
source .venv/bin/activate
python test_peer_revocation_regressions.py
python test_message_receipts.py
python test_efficient_revocation.py
python test_modular_crypto.py
python test_dpki.py
python test_network_e2e.py
```

The regression suite covers stale caches, reinstatement, TTL expiry, forged transactions, signature checks, delayed and missing chain replies, shorter forks, bootstrap reconnection without disrupting direct state, and a real three-node revoke/block/reinstate/resume sequence. test_network_e2e.py runs the direct-mesh suite in test_direct_mesh.py: discovery refuses message forwarding, broadcasts arrive once, and messaging, receipts, key lifecycle operations, chain refresh, reconnection and peer exchange work after the bootstrap is stopped.

The scoped benchmark uses four peers in separate groups and checks packet counts without manually injecting peer keys or channel memberships.

These scripts start isolated localhost test relays. They check application behavior but do not prove that separate LAN devices can communicate. Complete the manual LAN tests in step 8 as well. The network scripts use automatically assigned ports and benchmark/cryptography test ports 4099 and 4015; they do not replace the live bootstrap on 4001.

Troubleshooting on Debian

Symptom	Check and next step
python: command not found	Activate .venv; create it with python3 -m venv .venv if needed.
externally-managed-environment	Run pip after source .venv/bin/activate. which python should point into this project's .venv/bin/.
ensurepip unavailable or virtual environment creation fails	Install python3-venv, then run python3 -m venv .venv again.
ModuleNotFoundError: cryptography	In the active environment, run python -m pip install -r requirements.txt.
Missing peer_cli, peer_node, or another project module	Copy the complete current Python source and run from p2p-simple. A stale Git clone may lack the CLI changes.
Failed wheel build or unsupported Rust	Upgrade pip inside .venv; see the source-build instructions in step 3.
nc reports connection refused	Start the relay, verify its IP/port, and check ss -ltnp 'sport = :4001' on the relay device. Check for firewall rejection as well.

Symptom	Check and next step
nc times out	Check the subnet, route, relay firewall, AP/client isolation, and any VM networking. The remote endpoint must be the relay's LAN IP.
Relay cannot bind because port 4001 is in use	Check the listener with ss; use the intended existing relay or select another RELAY_PORT and update every peer endpoint and firewall rule.
Bootstrap is unavailable	Existing connections continue. For initial discovery, start it or use --no-relay --peer HOST:PORT with a known peer listener.
Discovery works but peers stays empty	Check each peer's --advertise-host, inbound TCP 5001 and AP/client isolation. On each device run ss -ltnp 'sport = :5001'; test from another device with nc -4 -vz -w 3 PEER_IP 5001.
Peer listener cannot start	Choose an unused --listen-port and a local --listen-host; multiple peers on one device need distinct ports.
A peer repeatedly disconnects when another launches	Check that the two processes are not sharing a copied identity JSON file. Relaunch the affected test peer with an unused --id-file name.
DM target is not found	Use the peer ID from peers, rather than its IP address. Copy the full ID to avoid an ambiguous short prefix.
Channel decryption fails	Match crypto settings on all participants using crypto sha256 aes-gcm; verify that each receiver joined the channel.
Node 3 receives a revocation push during Test A	It may still share #general or another channel, or have DM history with Node 2. Restart peers and perform Test A before the other messaging tests.
Node 3 reports a cache hit instead of the expected first miss	Start with fresh peer processes for Test A. A previous refresh or message may already have populated the cache.
SYNC_UNAVAILABLE or a chain request timeout	Confirm both peers are connected and use the same current source. Run refresh <peerId> once the network is responsive.
Icons appear as squares	Use a UTF-8 terminal with suitable fonts; judge tests by message text, connection state, and metrics.

Shut down and restart

Enter exit in each peer CLI, then press Ctrl+C in the relay terminal.

To restart, activate the environment in every terminal and rerun the peer commands

from step 7. Supply a running bootstrap or use a known peer address. Run deactivate in a

Linux shell when you have finished using its virtual environment.

Execution record

Record the date, device names, operating systems, Python versions, LAN addresses, peer IDs and source revision. For each test, record the sender command, receiver output and pass or fail result. Attach terminal captures showing messages and a successful refresh while the bootstrap remains stopped.